ComicCraft: AI Comic Story Creator using Gemini Models

Project Documentation

1. Project Description

ComicCraft is an AI-powered comic story creator that converts a user's story idea into a five-panel illustrated comic. The application accepts a story prompt, main character, setting, story tone and art style. It uses generative AI to create the story and AI-generated illustrations, displays the comic as a preview, and provides a downloadable PDF.

2. Scenarios

Scenario 1: A user enters a simple children's story idea, selects a main character and setting, and generates a five-panel comic.

Scenario 2: A user provides a character and story prompt and selects an art style to create illustrated comic panels.

Scenario 3: A user generates the comic, reviews the panel preview, and downloads the final comic as a PDF.

3. Technical Architecture

ComicCraft follows a modular architecture. The frontend collects user inputs and displays the generated comic. The FastAPI backend handles routing, processing and integration with AI services. Gemini is used for story generation and Hugging Face Inference Providers with FLUX.1-schnell are used for image generation. The generated panels are assembled into a comic preview and PDF.

Architecture Flow:

User → Frontend → FastAPI Backend → Gemini / Hugging Face → Comic Panels → Preview → PDF

4. Pre-requisites

- Windows operating system
- Python 3.x
- Visual Studio Code
- Internet connection
- Gemini API key
- Hugging Face API key
- Required Python packages from requirements.txt

5. Project Workflow

Milestone 1: Model Selection and Architecture

- Configure Gemini and Hugging Face API access.

- Select AI models suitable for story and image generation.
- Define the frontend, FastAPI backend and AI integration architecture.

Milestone 2: Core Functionalities Development

- Generate a five-panel story outline.
- Generate story content with narration and dialogue.
- Generate AI illustrations for each panel.

Milestone 3: Backend Development

- Implement FastAPI routes.
- Connect story generation and image generation modules.
- Create comic preview and PDF export functionality.

Milestone 4: Frontend Development

- Create the ComicCraft input interface.
- Display generated panels, scenes, actions, narration and dialogue.
- Provide PDF download functionality.

Milestone 5: Local Deployment and Testing

- Run the application using Uvicorn.
- Test story generation, image generation, preview and PDF download.
- Verify the FastAPI documentation endpoint.

Milestone 6: Conclusion

- Complete the working local ComicCraft application and prepare project documentation and demonstration.

6. Model Selection and Architecture

Gemini is used as the generative AI service for creating structured story content. Hugging Face Inference Providers with FLUX.1-schnell are used to create the comic illustrations. The application keeps API credentials in environment variables rather than hard-coding them.

7. Core Functionalities Development

- Story outline generation for five comic panels.
- Story generation with scene, action, narration and dialogue.
- AI image generation for individual panels.
- Comic layout and preview generation.
- PDF export and download.
- FastAPI interactive API documentation through /docs.

8. Backend Development

Main application components include:

- app/main.py – starts the FastAPI application.
- app/routes.py – handles application routes and comic generation workflow.

- app/gemini_flash.py – supports Gemini story generation.
- app/gemini_pro.py – contains supporting Gemini functionality.
- app/image_generator.py – generates AI comic illustrations using Hugging Face.
- app/layout_builder.py – handles comic layout generation.
- app/exporters.py – supports export functionality.

Important application routes:

- GET / – ComicCraft home page.
- POST /generate – generates the comic workflow.
- POST /generate-comic/json – JSON comic generation endpoint.
- GET /test-image – image generation test page.
- GET /download/{filename} – downloads generated files.
- GET /export-success – export success page.
- GET /docs – FastAPI API documentation.

9. Frontend Development

The ComicCraft frontend provides fields for Story Prompt, Main Character Name, Setting, Story Tone and Art Style. The user starts generation using the Generate My Comic button. The generated five-panel comic is then displayed with AI illustrations and story information, followed by a PDF download option.

10. Local Deployment and Testing

Run the project from the project folder using:

```
..\env\Scripts\python.exe -m uvicorn app.main:app --reload
```

Open the application at:

```
http://127.0.0.1:8000
```

Open the FastAPI documentation at:

```
http://127.0.0.1:8000/docs
```

11. Testing and Verification

- Home page loading – Passed.
- Story generation – Passed.
- Five-panel generation – Passed.
- AI image generation – Passed.
- Comic preview – Passed.
- PDF generation – Passed.
- PDF download – Passed.
- FastAPI /docs access – Passed.

12. Project File Structure

```
ComicCraft_Full_Project/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── routes.py
│   ├── gemini_flash.py
│   ├── gemini_pro.py
│   ├── image_generator.py
│   ├── layout_builder.py
│   └── exporters.py
├── templates/
│   ├── index.html
│   ├── comic_preview.html
│   ├── export_success.html
│   └── test_image.html
├── static/
│   ├── css/
│   ├── panels/
│   └── exports/
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

13. Deployment / Access Details

Hosted / Deployed URL	Not currently hosted; runs locally using FastAPI and Uvicorn.
Login Credentials	Not required.
Platform / Hosting Provider	Local development environment – FastAPI + Uvicorn.
Repository Link	https://github.com/vasilabanuvasilabanu/ComicCraft-AI-Comic-Story-Creator
Demo Video Link	To be added after uploading the project demo video.

14. Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The application currently runs locally and is not hosted online.	Run the application using FastAPI and Uvicorn on the local system.
2	AI story and image generation require valid API keys and internet connection.	Configure GEMINI_API_KEY and HF_API_KEY in the .env file.
3	AI image generation depends on Hugging Face service availability and usage limits.	Ensure the Hugging Face token has the required permissions and retry if the service is temporarily unavailable.

15. Conclusion

ComicCraft demonstrates an end-to-end generative AI workflow for creating illustrated comics from user-provided story ideas. The project combines FastAPI, Gemini-based story generation and Hugging Face image generation, followed by comic preview and PDF export. The completed application provides a simple workflow for creating and downloading a five-panel AI comic.

Project Commands and Picture References

The following command references and visual guides are included to make the ComicCraft execution process easy to understand and reproduce.

Picture 1: Opening the Project Terminal

Open the ComicCraft_Full_Project folder in VS Code and use the integrated terminal.

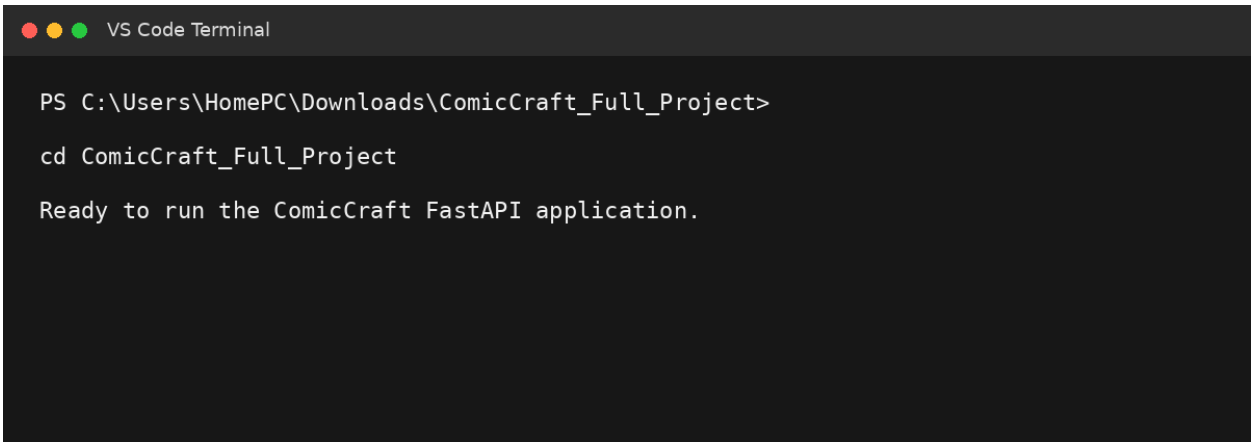


Figure 1 – VS Code terminal and ComicCraft project folder.

Picture 2: Command to Run ComicCraft

Run the FastAPI application using the following command:

```
..\env\Scripts\python.exe -m uvicorn app.main:app --reload
```

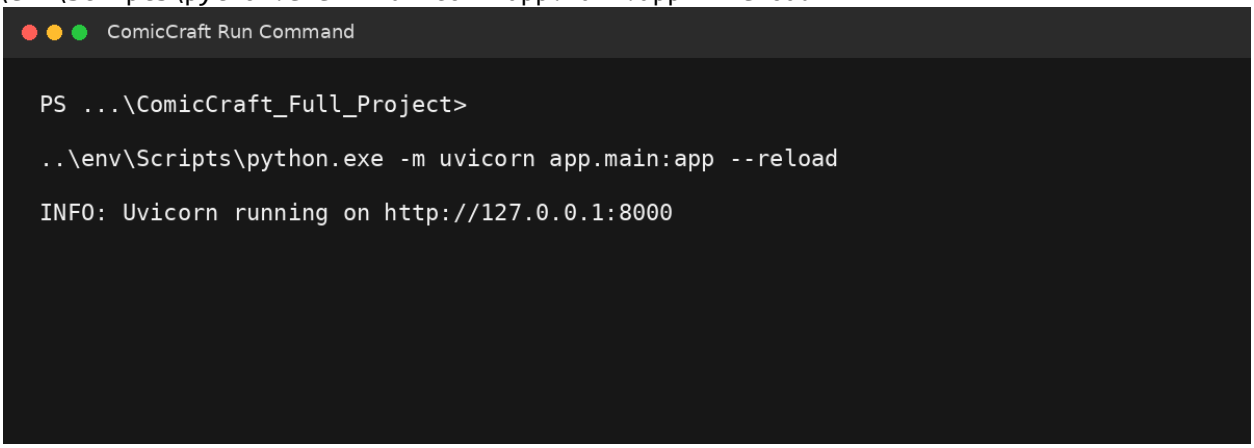


Figure 2 – ComicCraft FastAPI/Uvicorn run command.

Picture 3: ComicCraft Home Page

After the server starts, open the local application in a browser.



Figure 3 – ComicCraft local homepage and input fields.

Picture 4: FastAPI /docs Page

The API endpoints can be checked using the FastAPI documentation page.

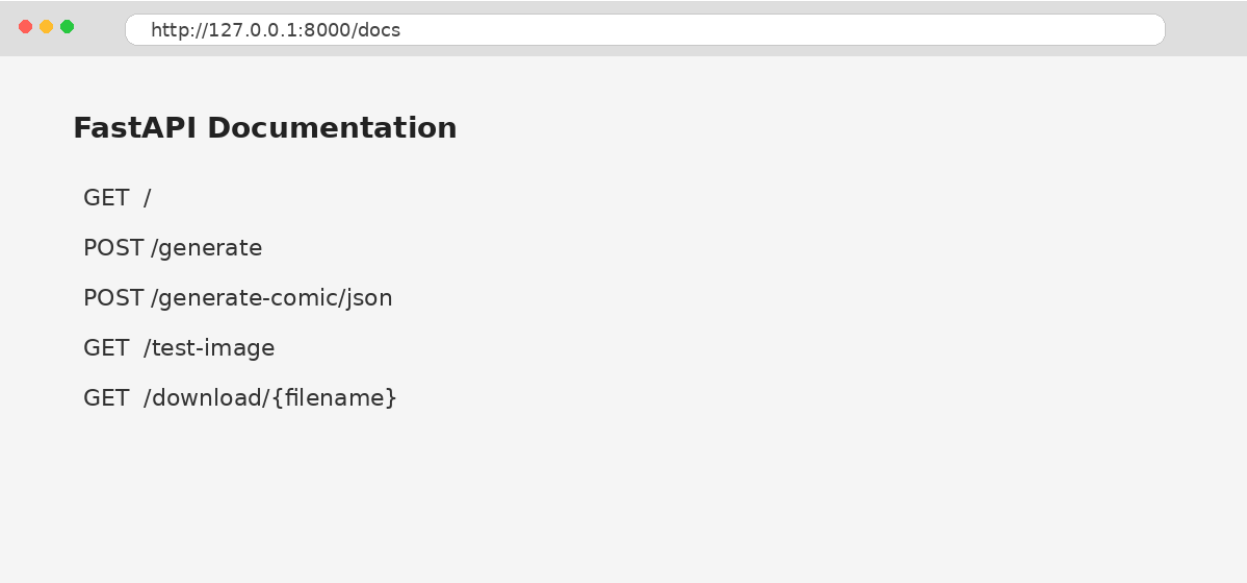


Figure 4 – ComicCraft FastAPI API documentation.

Picture 5: ComicCraft Architecture

The application accepts user input through the frontend, processes it through FastAPI, uses Gemini for story generation, and uses Hugging Face for AI image generation.

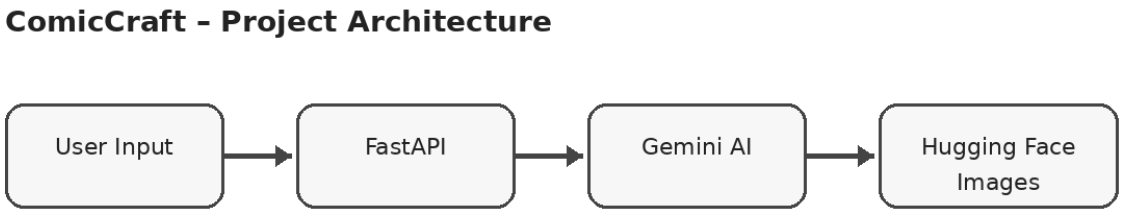
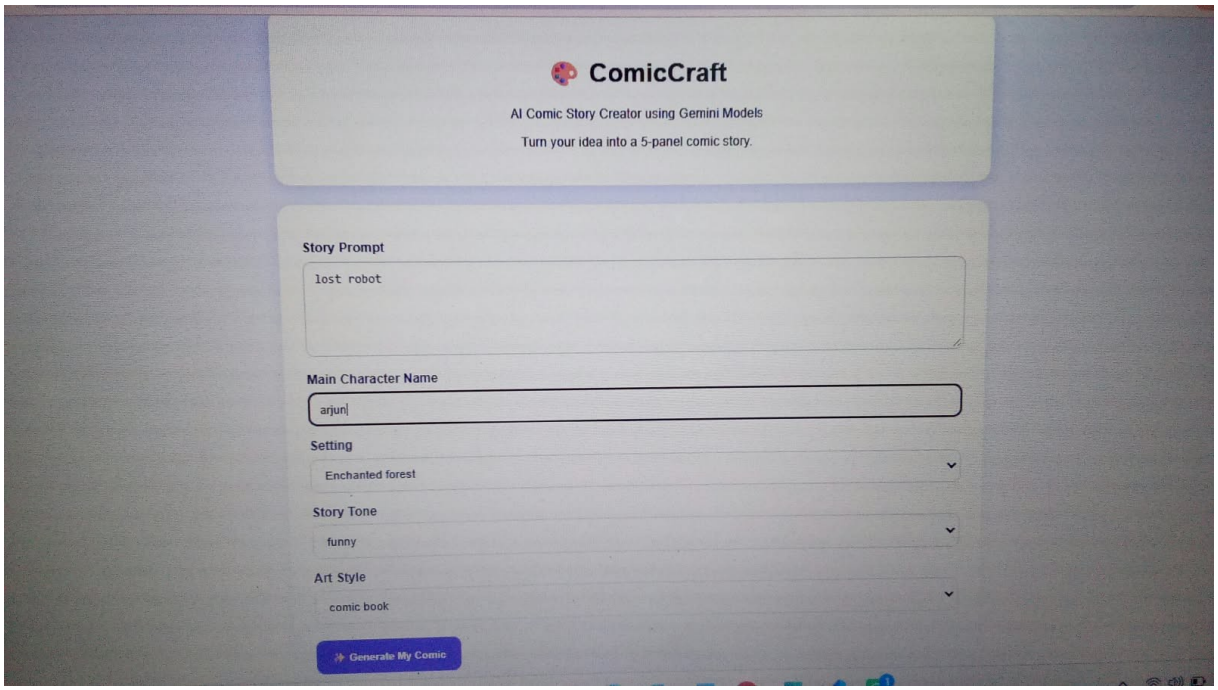


Figure 5 – Simplified ComicCraft architecture flow.

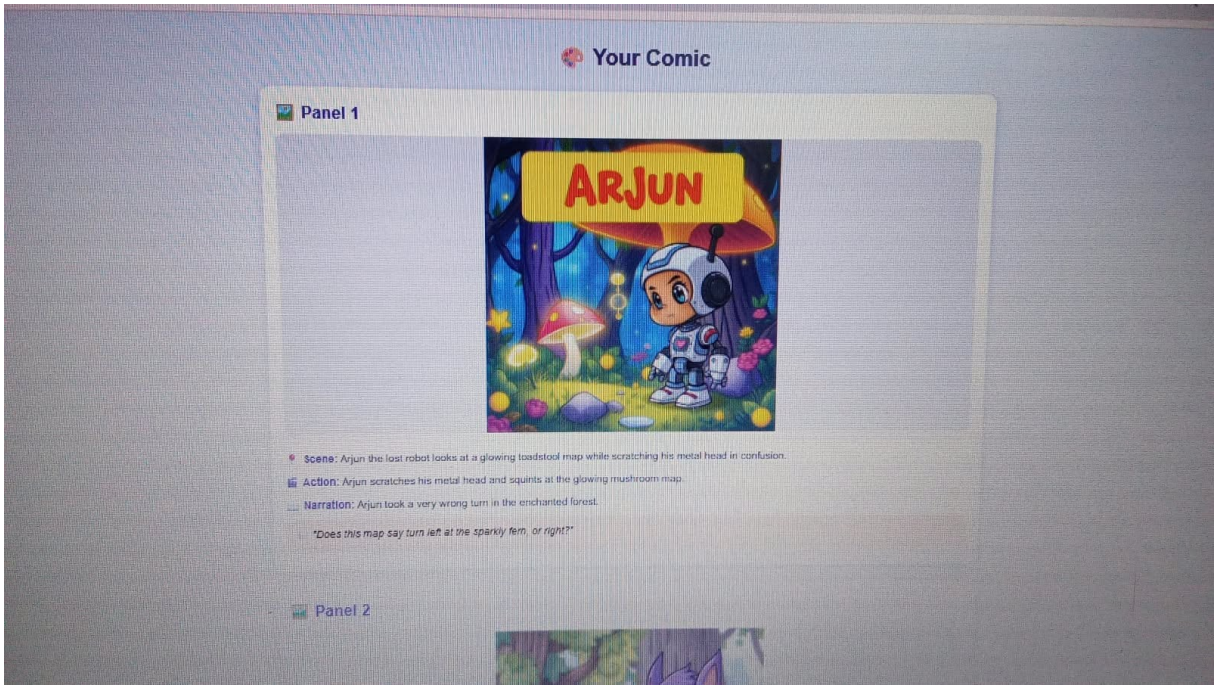
Project Output Screenshots

The following screenshots show the actual ComicCraft application output and user interface captured during project execution.

Picture 1: ComicCraft Input Page



Picture 2: Generated Comic Output – Panel 1 and Panel 2



Picture 3: Generated Comic Output – Panel 5 and PDF Download

